PROJET PWA & ARCHITECTURE WEB AVANCÉE

SmartApply Tunisia — Fiche Mémo Soutenance

Guide complet : Architecture PWA, Gestion des Rôles, Workflow & Stack Technique

1. Fiche d'Identité & Objectifs du Projet

SmartApply Tunisia est une application web moderne conçue comme une **Progressive Web App (PWA)** dédiée au recrutement IT en Tunisie (stages PFE, premier emploi, opportunités tech). Elle relie trois acteurs clés : les **Candidats** (chercheurs d'emploi/étudiants), les **Entreprises** (recruteurs partenaires) et les **Super Administrateurs** qui supervisent l'écosystème avec validation des structures.

Composant	Choix Technologique	Justification Technique
Framework Frontend	Next.js 14 (App Router) + React 18	Server Components (RSC) pour le SEO, performances et rendu hybride.
Langage	TypeScript 5	Typage statique strict réduisant les erreurs d'exécution à zéro.
Design & UI	Tailwind CSS	Design System moderne SaaS, mobile-first et utilitaires responsives légers.
Backend & BDD	Supabase (PostgreSQL 15 Cloud)	Base relationnelle robuste avec Row Level Security (RLS) et Auth JWT.
Gestion de Session	@supabase/ssr + Cookies chiffrés	Authentification sécurisée compatible SSR et Edge Middleware.
Architecture PWA	Service Worker + Web App Manifest	Mode hors ligne, mise en cache et installation native sur OS/mobile.

2. Le Principe de la Progressive Web App (PWA)

Une **PWA** est une application web qui exploite les standards modernes du navigateur pour offrir une expérience utilisateur identique à une application mobile native (fluidité, installation, fonctionnement sans connexion Internet), tout en restant accessible via une simple URL.

1. Web App Manifest (manifest.json)

Fichier JSON déclarant les métadonnées de l'application : nom officiel, couleurs de thème (#2563EB), icônes standardisées (192x192 et 512x512) avec propriété *any maskable*, et affichage en mode **standalone** (lancement en plein écran sans barre d'adresse navigateur).

2. Service Worker (sw.js)

Script JavaScript exécuté en arrière-plan par le navigateur sur un thread séparé. Il agit comme un **proxy réseau côté client** : il intercepte chaque requête sortante (`fetch`), consulte le cache local (`Cache Storage`) et décide d'interroger le réseau ou de renvoyer le cache.

3. Mode Hors Ligne (Offline)

• **Stratégie Cache First :** Appliquée aux assets critiques (CSS, JS, icônes) pour un chargement instantané.
• **Stratégie Network First avec Fallback :** Pour les pages HTML. Si le réseau est indisponible, le Service Worker sert la page mise en cache ou bascule automatiquement sur la page dédiée **offline.html**.
• **Indicateur dynamique :** Le composant OfflineIndicator.tsx surveille window.onLine et informe l'utilisateur en temps réel de la perte ou du rétablissement de la connexion.

4. Ergonomie Mobile-First & Responsive

Interface pensée prioritairement pour smartphone : balise viewport adaptée, sidebar rétractable en drawer avec fond flouté, zones tactiles de minimum 44px, et grilles Tailwind fluides (1 col mobile ➔ 4 cols desktop).

3. Gestion des Rôles (RBAC) & Sécurité

SmartApply implémente un contrôle d'accès basé sur les rôles (**Role-Based Access Control - RBAC**). L'authentification est découplée en deux couches : l'identité dans auth.users et le profil applicatif dans la table public.profiles.

Rôle	Accès & Dashboard	Responsabilités & Fonctionnalités
Candidat [Étudiant / Diplômé]	`/dashboard/candidate`	Inscription publique directe. Consultation des offres publiées, accès à l'annuaire des entreprises vérifiées, suivi des candidatures et paramètres de profil.
Entreprise [Recruteur partenaire]	`/dashboard/company`	Inscription avec nom de société. Statut initial **pending** (en attente). Publication d'offres (CDI, Stage PFE, CDD), modification et suppression de ses offres, édition de la fiche société.
Super Admin [Superviseur]	`/dashboard/admin`	Compte unique protégé (non disponible à l'inscription publique). Tour de contrôle globale : modération des entreprises (Valider/Rejeter), supervision et suppression des utilisateurs/candidats, gestion des offres et rapports.

4. Architecture & Workflow Utilisateur

1. Inscription : L'utilisateur choisit son rôle (Candidat ou Entreprise). Le Server Action register() enregistre le compte dans Supabase auth.users, insère le profil dans public.profiles avec son nom et email, puis ouvre immédiatement une session sans bloquer l'utilisateur.

2. Protection par Middleware (middleware.ts) : À chaque requête de navigation vers /dashboard/*, le middleware Edge intercepte la demande, lit la session chiffrée, extrait le rôle de l'utilisateur et bloque tout accès illégitime (ex. un candidat tentant d'accéder à /dashboard/admin est redirigé vers /access-denied). Le Super Admin a accès à toutes les zones.

3. Modération Entreprise : Lorsqu'une entreprise s'inscrit, elle apparaît dans l'espace modération de l'Administrateur avec un badge jaune *En attente*. L'administrateur peut en un clic valider (statut verified) ou rejeter le dossier.

4. Affichage Intelligent des Candidats : La page /dashboard/admin/users applique une pagination côté serveur (via range(from, to)) pour garantir une vitesse instantanée même en cas de plusieurs milliers de candidats inscrits.

5. Questions Fréquentes du Jury & Réponses Idéales

Anticipez les questions techniques de votre enseignant avec ces réponses synthétiques :

Q1 : Qu'est-ce qui prouve concrètement que votre application est une PWA ?

Réponse : Trois preuves techniques indiscutables :
1. La présence du fichier manifest.json avec ses icônes 192x192 et 512x512 permettant l'installation via le bouton d'installation du navigateur.
2. L'enregistrement actif d'un Service Worker vérifiable dans l'onglet *Application* des DevTools Chrome.
3. Le fonctionnement en mode déconnecté grâce au Cache Storage et la page de secours offline.html.

Q2 : Pourquoi avoir utilisé Next.js 14 plutôt qu'un React classique (CRA/Vite) ?

Réponse : Next.js 14 apporte le rendu hybride (Server Components), un routage basé sur les dossiers (App Router), une sécurité accrue avec les **Server Actions** (le code sensible et les clés d'API ne transitent jamais sur le client), et un meilleur référencement (SEO) indispensable pour une plateforme d'emploi.

Q3 : Comment garantissez-vous la sécurité des données entre les rôles ?

Réponse : Nous appliquons une sécurité à double niveau :
• Au niveau de l'application : Le middleware.ts filtre les routes selon le rôle authentifié par token JWT.
• Au niveau de la base de données : Supabase applique les **Row Level Security (RLS)**. Même en cas de requête directe sur la base, un utilisateur ne peut manipuler que ses propres données, tandis que l'administrateur dispose de droits de supervision étendus.

Q4 : Quelle stratégie de mise en cache avez-vous adoptée dans le Service Worker ?

Réponse : Une stratégie hybride :
• **Cache First :** Pour les ressources statiques immuables (images, polices, styles) afin d'accélérer l'affichage.
• **Network First :** Pour les pages de navigation dynamique (offres, profil) afin de toujours afficher les données fraîches lorsque le réseau est présent, avec basculement automatique sur le cache ou offline.html en cas de déconnexion.

Q5 : Comment la base de données sépare-t-elle les candidats et les entreprises ?

Réponse : Tous les utilisateurs possèdent un identifiant unique (UUID) dans auth.users. La table public.profiles stocke leur rôle (candidat, entreprise, admin). Lorsqu'un compte entreprise est créé, une ligne correspondante est insérée dans la table public.companies avec un statut pending jusqu'à sa validation par l'administrateur.

Q6 : Comment le responsive design a-t-il été testé et mis en place ?

Réponse : Grâce à l'approche **Mobile-First** de Tailwind CSS : la disposition par défaut cible les téléphones portables (sidebar en tiroir coulissant, tableaux défilants horizontalement), puis s'enrichit progressivement via les breakpoints sm:, md: et lg: sur ordinateur.

■ **Conseil pour votre présentation orale :** Montrez d'abord la création d'un compte Candidat, puis montrez l'espace Super Admin avec la modération des entreprises, et terminez en coupant la connexion dans l'onglet *Network* pour prouver la PWA en direct !